UNIT – I - SQL Vs. SQL*Plus

Introduction

Before learning SQL commands, it is important to understand the difference between **SQL** and **SQL*Plus**. Most beginners think they are the same because both are used while working with databases. However, they have different purposes.

- **SQL** is a language used to communicate with a relational database.
- **SQL*Plus** is a software tool provided by Oracle that allows users to execute SQL and PL/SQL commands.

In simple words, **SQL tells the database what to do, while SQL*Plus provides the environment to write and execute those SQL commands.**

Learning Objectives

After studying this chapter, you will be able to:

- Understand the meaning of SQL.
- Explain SQL*Plus.
- Differentiate between SQL and SQL*Plus.
- Understand why SQL is important.
- Identify the features and applications of SQL.
- Understand where SQL*Plus is used.

What is SQL?

SQL stands for **Structured Query Language**.

It is the standard language used to communicate with relational databases. SQL allows users to create databases, store data, retrieve data, update records, and delete unwanted information.

Almost every Relational Database Management System (RDBMS) such as **MySQL, Oracle, SQL Server, PostgreSQL, and SQLite** supports SQL.

Definition

SQL is a standard language used to create, access, manipulate, and manage data stored in relational databases.

Why Do We Need SQL?

Imagine a college stores information about thousands of students.

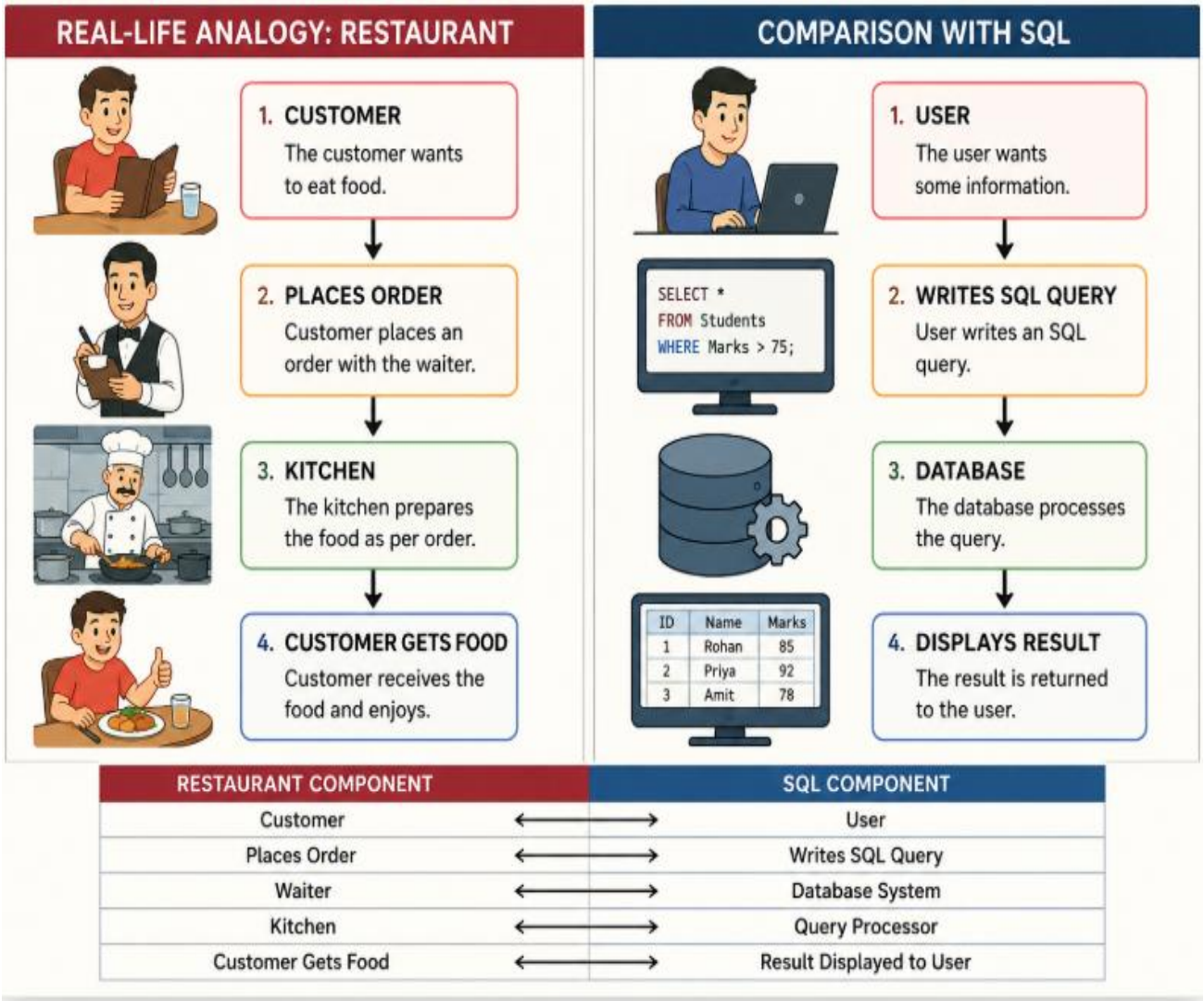
The database contains:

- Student Name
- Roll Number
- Course
- Semester
- Marks
- Address
- Mobile Number

Suppose the principal wants to know:

- Which students scored above 80 marks?
- How many students are studying in BCA?
- Which students belong to Jaipur?

The database cannot understand English sentences.
Instead, we use SQL.
Example:
SELECT * FROM Student;
This command tells the database to display all records from the Student table.



Restaurant Database

Customer	User
Waiter	SQL
Kitchen	Database
Food	Result

Just as the waiter communicates with the kitchen, SQL communicates with the database.

History of SQL

The history of SQL is interesting.

1970

Dr. Edgar F. Codd, working at IBM, introduced the **Relational Database Model**.

1974

IBM developed a language called **SEQUEL (Structured English Query Language).**

1979

Oracle released the first commercial relational database using SQL.

1986

ANSI adopted SQL as an international standard.

1987

ISO also approved SQL as an international database language.

Today, SQL is used in almost every organization around the world.

Features of SQL

1. Easy to Learn: SQL uses simple English-like commands.

Examples:

SELECT

INSERT

UPDATE

DELETE

Students can learn SQL quickly because its syntax is easy to understand.

---

2. Standard Language: SQL follows ANSI and ISO standards.

This means SQL knowledge can be used with different databases.

Examples:

- MySQL
- Oracle
- SQL Server
- PostgreSQL

---

3. Portable: A SQL query written for one database can often be used in another with only minor changes.

---

4. Fast: SQL can retrieve millions of records within seconds.

Example: A bank with 50 lakh customer records can quickly find one customer's account details using SQL.

---

5. Secure: SQL allows administrators to control user permissions.

Example:

- Teacher → Can update marks.
- Student → Can only view marks.

---

6. Powerful

SQL can perform many operations, such as:

- Creating databases
- Creating tables
- Inserting records
- Updating records
- Deleting records
- Searching records
- Sorting records
- Grouping records

Applications of SQL

SQL is used in almost every industry.

Banking

- Customer accounts
- ATM transactions
- Loan management

Education

- Student records
- Attendance
- Examination system

Hospital

- Patient records
- Doctor schedules
- Medical reports

Railway Reservation

- Ticket booking
- Passenger details
- Train schedules

E-Commerce

- Product details
- Orders
- Customer information

Social Media

- User profiles
- Posts
- Comments
- Likes

Government

- Citizen records
- Passport information
- Aadhaar database

Advantages of SQL

- Easy to understand
- Saves time
- Reduces manual work
- Handles large amounts of data
- Supports multiple users
- Highly secure
- Widely accepted
- Supports complex queries

Limitations of SQL

Although SQL is very powerful, it has some limitations.

- SQL cannot perform complex programming tasks like Java or Python.
- It does not support loops and conditional statements as effectively as programming languages (these are handled by PL/SQL or procedural extensions).
- Different database systems may have slight differences in SQL syntax.

What is SQL*Plus?

SQL*Plus is a **command-line interface (CLI)** tool developed by Oracle.
It provides an environment where users can write and execute SQL and PL/SQL commands.
SQL*Plus itself is **not a database**. It acts as a bridge between the user and the Oracle Database.

Definition

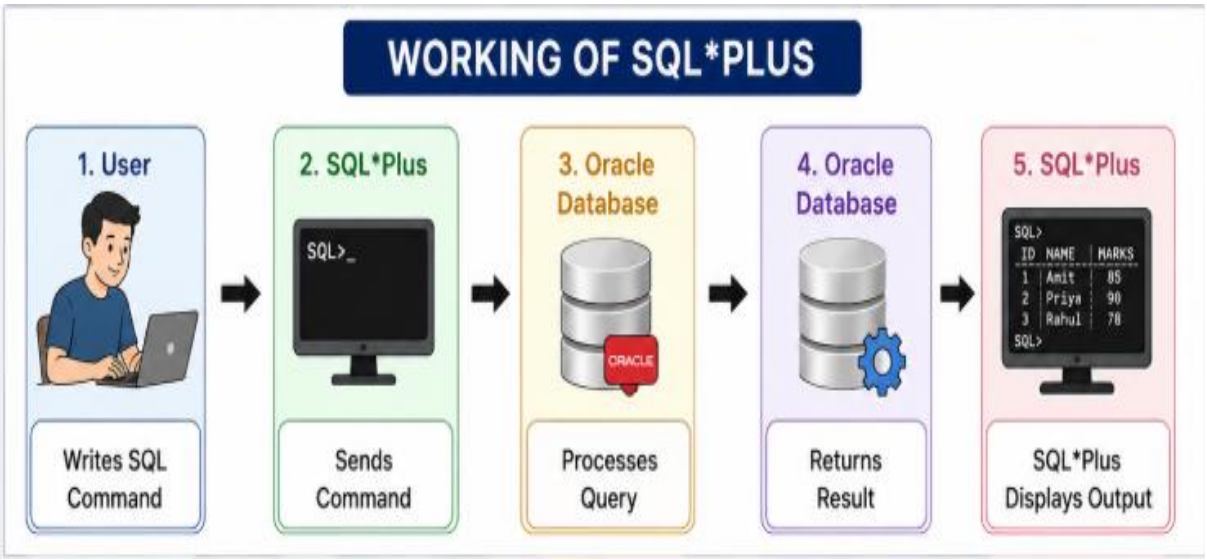
SQL*Plus is an Oracle command-line tool used to execute SQL and PL/SQL statements and display the results.

Why Do We Need SQL*Plus?

Suppose you write this SQL query:
SELECT * FROM Student;
The query needs an application to send it to the Oracle database.
SQL*Plus performs this task.
It:

- Accepts SQL commands from the user.
- Sends them to the Oracle Database.
- Receives the result.
- Displays the output on the screen.

Working of SQL*Plus



Features of SQL*Plus

- Executes SQL statements
- Executes PL/SQL programs
- Saves SQL scripts
- Runs SQL script files
- Formats query output
- Generates reports
- Displays error messages
- Connects users to Oracle Database

SQL vs SQL*Plus:

SQL	**SQL*Plus**
SQL stands for Structured Query Language.	SQL*Plus is an Oracle command-line tool.
It is a language.	It is software.
Used to communicate with databases.	Used to execute SQL and PL/SQL commands.
Standardized by ANSI/ISO.	Developed by Oracle Corporation.
Works with MySQL, Oracle, SQL Server, PostgreSQL, etc.	Works primarily with Oracle Database.
Cannot execute scripts by itself.	Can execute SQL scripts and PL/SQL programs.
Used for database operations.	Used for interacting with the Oracle database.

Key Points to Remember

- SQL is a **language**, while SQL*Plus is a **tool**.
- SQL is used to create, retrieve, update, and delete data.
- SQL*Plus provides the environment to execute SQL commands in Oracle.
- SQL follows ANSI/ISO standards.
- SQL is supported by almost all relational databases.
- SQL*Plus is specific to Oracle Database.

Introduction of SQL Commands:

A database contains a large amount of data. To work with this data, we need instructions that tell the database what to do. These instructions are called **SQL Commands**.

An SQL command can be used to:

- Create a database
- Create tables
- Insert data
- Retrieve data
- Update data
- Delete data
- Control user access
- Manage transactions

In simple words, **SQL commands are instructions that help users communicate with the database.**

What is an SQL Command?

Definition

An SQL command is an instruction written in SQL that performs a specific operation on a database or its objects.

For example,

CREATE DATABASE College;

This command creates a new database named **College**.

Another example,

SELECT * FROM Student;

This command displays all records from the Student table.

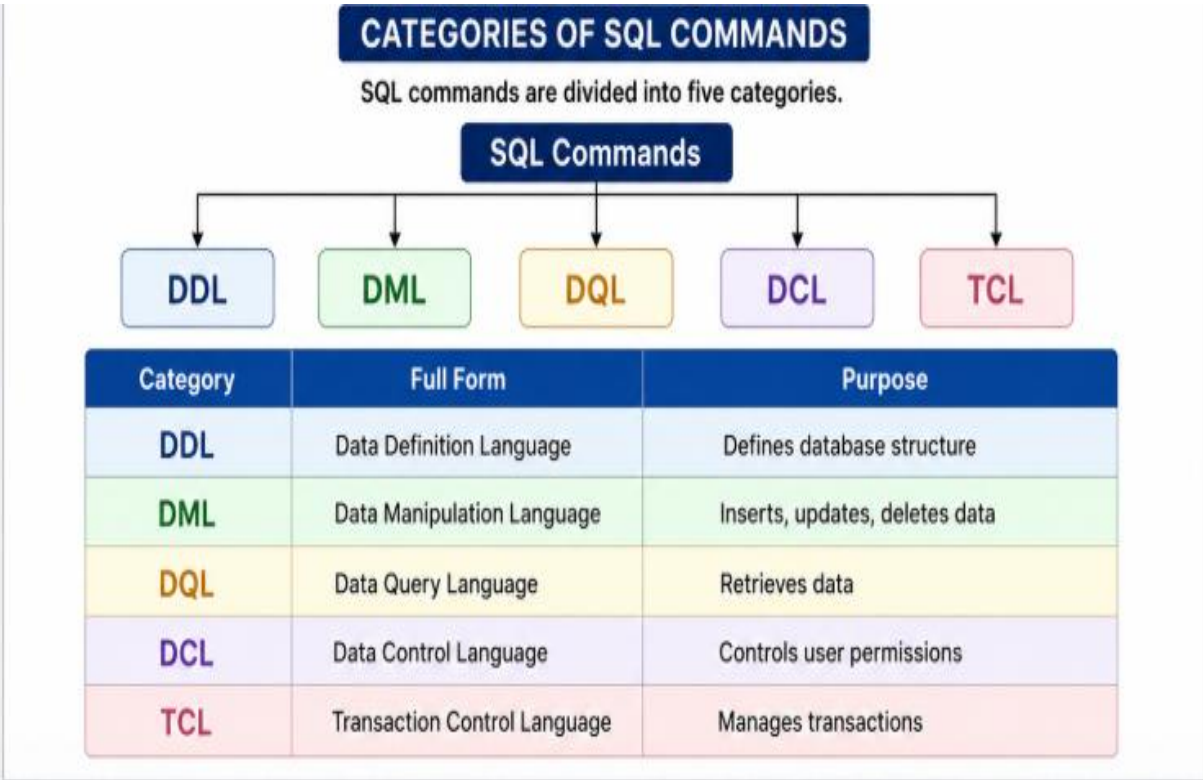
Why Do We Need SQL Commands?

Imagine a college has a database containing 10,000 student records.

A teacher wants to:

- Add a new student.
- Update marks.
- Find students who scored above 80.
- Delete records of graduated students.

Without SQL commands, these tasks would not be possible.



Category	Full Form	Purpose
DDL	Data Definition Language	Defines database structure
DML	Data Manipulation Language	Inserts, updates, deletes data
DQL	Data Query Language	Retrieves data
DCL	Data Control Language	Controls user permissions
TCL	Transaction Control Language	Manages transactions

---

1. DDL (Data Definition Language)

Definition

DDL stands for **Data Definition Language**.

It is used to create and modify the structure of database objects.

DDL works on:

- Database
- Tables
- Views
- Indexes

It changes the **structure** of the database, **not the data**.

Characteristics of DDL

- Defines database objects.
- Automatically saves changes (Auto Commit).
- Cannot be rolled back.
- Changes table structure.

Common DDL Commands

- CREATE
- ALTER
- DROP
- TRUNCATE
- RENAME

CREATE Command

The CREATE command creates a new database or table.

Syntax

```
CREATE TABLE table_name
(
column1 datatype,
column2 datatype
);
```

Example

```
CREATE TABLE Student
(
RollNo INT,
Name VARCHAR(50),
Marks INT
);
```

This command creates a Student table.

ALTER Command

Used to modify an existing table.

Example:

Add a column.

```
ALTER TABLE Student
ADD Address VARCHAR(100);
```

Result

A new column **Address** is added.

DROP Command

Removes an entire table permanently.

Syntax

```
DROP TABLE Student;
```

Result

The Student table and all its records are deleted permanently.

⚠️ Cannot be recovered.

TRUNCATE Command

Deletes all records but keeps the table structure.

Example

```
TRUNCATE TABLE Student;
```

Before

Roll	Name
101	Rahul
102	Priya

After
Table exists.
Records = 0

RENAME Command
Changes the table name.
RENAME TABLE Student TO Student_Details;

Real-Life Example of DDL
Suppose a school is opening a new library.
First, they build:

- Book shelves
- Reading room
- Issue counter

No books have been placed yet.
Similarly,
DDL builds the database structure.

---

2. DML (Data Manipulation Language)
Definition
DML stands for **Data Manipulation Language**.
It is used to manipulate the data stored inside tables.
Unlike DDL,
DML works on **rows (records)** rather than the structure.

DML Commands

- INSERT
- UPDATE
- DELETE

INSERT Command
Used to insert new records.
Syntax
INSERT INTO Student
VALUES
(101,'Rahul',85);
Table

Roll	Name	Marks
101	Rahul	85

UPDATE Command
Changes existing data.
Syntax

UPDATE Student
SET Marks=90
WHERE RollNo=101;
Before
Rahul → 85
After
Rahul → 90

DELETE Command
Deletes selected records.
Example
DELETE FROM Student
WHERE RollNo=101;
Only one record is removed.
The table still exists.

Characteristics of DML

- Changes data.
- Can be rolled back before COMMIT.
- Works on table records.

Real-Life Example
Imagine a school register.
INSERT
New student admission.
UPDATE
Student changes address.
DELETE
Student leaves school.

---

3. DQL (Data Query Language)
Definition
DQL stands for **Data Query Language**.
It is used to retrieve information from a database.
The main command is
SELECT

SELECT Command
Syntax
SELECT * FROM Student;
Displays all records.

Specific columns
SELECT Name,Marks
FROM Student;
Output

Name	Marks
Rahul	90
Priya	85

Using WHERE
SELECT *
FROM Student
WHERE Marks>80;
Output
Only students scoring above 80.

Real-Life Example
A principal asks:
"Show all students whose marks are above 75."
SQL Answer
SELECT *
FROM Student
WHERE Marks>75;

---

4. DCL (Data Control Language)
Definition
DCL stands for **Data Control Language**.
It controls user permissions.
Usually used by Database Administrators (DBAs).

Commands

- GRANT
- REVOKE

GRANT
Provides permission.
Example
GRANT SELECT
ON Student
TO Teacher;
Teacher can now view student records.

REVOKE
Removes permission.

Example
REVOKE SELECT
ON Student
FROM Teacher;
Teacher can no longer view records.

Real-Life Example
College Library
Student gets Library Card
↓
GRANT Permission
Card cancelled
↓
REVOKE Permission

---

5. TCL (Transaction Control Language)
Definition
TCL stands for **Transaction Control Language**.
It manages transactions.
A transaction is a group of SQL statements executed together.

TCL Commands

- COMMIT
- ROLLBACK
- SAVEPOINT

COMMIT
Saves changes permanently.
Example
UPDATE Student
SET Marks=95
WHERE RollNo=101;
COMMIT;
Now changes cannot be undone.

ROLLBACK
Undo changes.
Example
UPDATE Student
SET Marks=95;
ROLLBACK;
Marks return to previous values.

SAVEPOINT
Creates a checkpoint.
Example
SAVEPOINT A;

UPDATE Student
SET Marks=90;

ROLLBACK TO A;
Returns to SAVEPOINT A.

Real-Life Example
ATM Transaction
Withdraw Money
↓
Cash Dispensed
↓
COMMIT
If ATM crashes before cash is dispensed
↓
ROLLBACK
No money is deducted.

Comparison of SQL Command Categories

Feature	DDL	DML	DQL	DCL	TCL
Purpose	Define Structure	Manipulate Data	Retrieve Data	Control Access	Manage Transactions
Works On	Tables	Records	Records	Users	Transactions
Example Commands	CREATE, ALTER	INSERT, UPDATE	SELECT	GRANT, REVOKE	COMMIT, ROLLBACK
Rollback Possible	No	Yes	Not Required	No	Yes

Difference Between DDL and DML

DDL	DML
Changes structure	Changes data
Auto Commit	Needs COMMIT
Cannot rollback	Can rollback
CREATE, ALTER	INSERT, UPDATE

Difference Between DML and DQL

DML	DQL
Modifies data	Retrieves data
INSERT, UPDATE	SELECT
Changes records	Only reads records

Key Points to Remember

- SQL commands are classified into **DDL, DML, DQL, DCL, and TCL**.
- **DDL** changes the structure of the database.
- **DML** inserts, updates, and deletes records.
- **DQL** retrieves data using the SELECT statement.
- **DCL** manages user permissions with GRANT and REVOKE.
- **TCL** ensures transaction consistency using COMMIT, ROLLBACK, and SAVEPOINT.

Introduction Data Types:

When we create a table in SQL, we need to specify what type of data each column will store. For example, a student's name contains letters, while age contains numbers, and date of birth contains dates. SQL uses **Data Types** to define the kind of data that can be stored in each column.

Choosing the correct data type is important because it:

- Saves storage space.
- Improves query performance.
- Maintains data accuracy.
- Prevents invalid data from being entered.

What is a Data Type?

Definition

A data type is an attribute that specifies the type of value a column can store in a database table.

For example:

- Student Name → Text
- Roll Number → Number
- Date of Birth → Date
- Salary → Decimal Number

Every column in a table must have a data type.

Why Do We Need Data Types?

Suppose we create a student table.

Roll No Name Age

The database must know:

- Roll Number should store numbers.

- Name should store text.
- Age should store numbers only.

If we accidentally store "Twenty" in the Age column, it will create incorrect data.
By assigning data types, SQL prevents such mistakes.

Example

```
CREATE TABLE Student
(
RollNo INT,
Name VARCHAR(50),
Age INT
);
```

Here,

- RollNo stores integers.
- Name stores text.
- Age stores integers.

Benefits of Using Data Types

1. Data Accuracy

Only valid data can be stored.
Example:
Age = 20 ✅
Age = "Twenty" ❌

---

2. Better Performance

Smaller data types require less memory.
Example:
Using SMALLINT instead of BIGINT for age saves storage.

---

3. Data Integrity

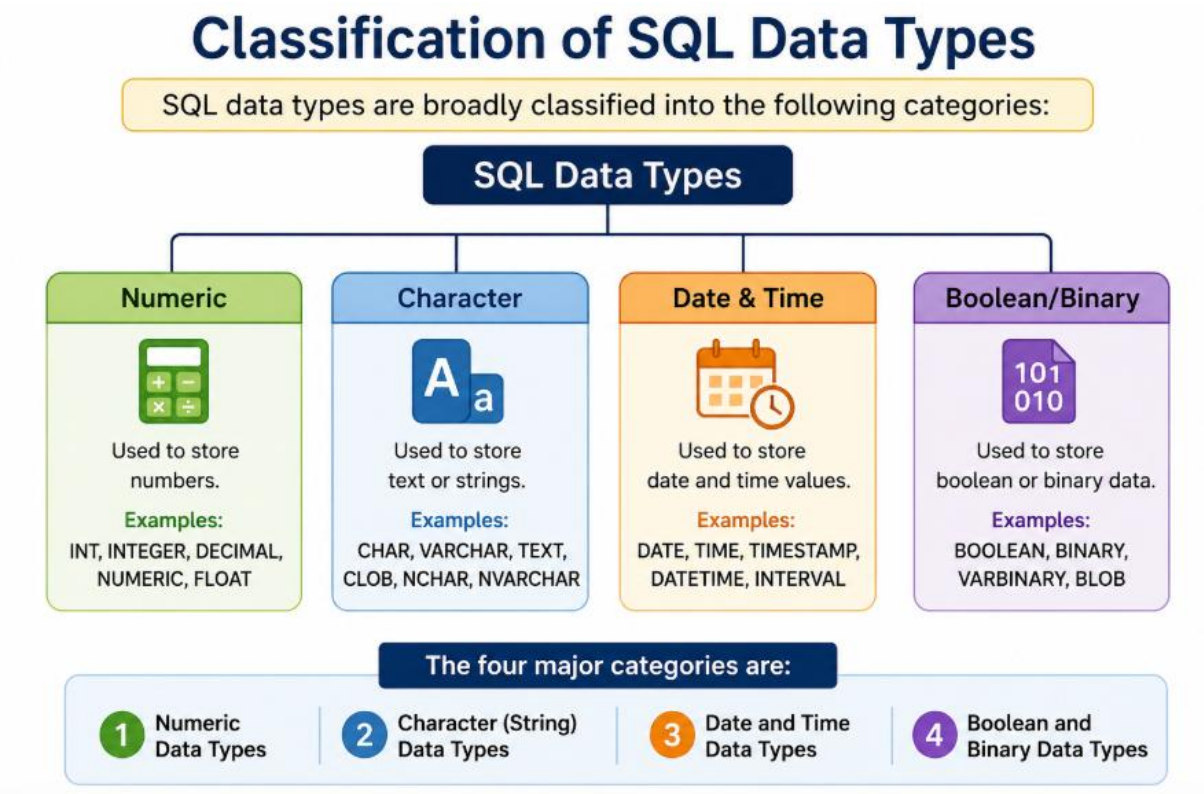
Data types ensure consistency in the database.
Example:
Date of Birth is always stored in the same format.

---

4. Efficient Storage

Correct data types reduce unnecessary storage usage.

Classification of SQL Data Types



1. Numeric Data Types

Numeric data types are used to store numbers.

These numbers may be:

- Whole numbers
- Decimal numbers
- Large numbers

INT

Stores whole numbers.

Example

Age INT

Possible values

18

25

100

Not Allowed

18.5

ABC

Uses

- Student ID
- Roll Number
- Quantity

SMALLINT
Stores small integer values.
Example
Age SMALLINT
Used when numbers are small.
Examples
Age
Class Strength
Semester Number

BIGINT
Stores very large integers.
Example
Population BIGINT
Used in

- Population
- Bank Account Numbers
- Aadhaar Numbers

DECIMAL
Stores exact decimal values.
Syntax
Salary DECIMAL(10,2)
Meaning
10 = Total digits
2 = Digits after decimal
Example
25000.75
99999.99

Uses

- Salary
- Product Price
- Bank Balance

FLOAT
Stores approximate decimal values.
Example
Temperature FLOAT
Used in

- Temperature
- Scientific Calculations
- GPS Coordinates

DOUBLE
Stores large decimal values with higher precision than FLOAT.
Example
Distance DOUBLE

Difference Between FLOAT and DECIMAL

FLOAT	DECIMAL
Approximate value	Exact value
Faster calculations	More accurate
Scientific data	Financial data
May cause rounding errors	No rounding errors

Example
Salary → DECIMAL ✅
Temperature → FLOAT ✅

---

2. Character (String) Data Types
These data types store text.
Examples
Student Name
City
Email
Address

CHAR
Stores fixed-length strings.
Example
Gender CHAR(1)
Possible Values
M
F
O
Length is always fixed.

VARCHAR
Stores variable-length strings.
Example
Name VARCHAR(50)
Possible Values
Rahul
Priya
Ananya Sharma
Only the required memory is used.

TEXT
Stores large amounts of text.
Example
Feedback TEXT
Uses

- Product Description
- Comments
- Reviews
- Feedback

Difference Between CHAR and VARCHAR

CHAR	VARCHAR
Fixed Length	Variable Length
Faster	Slightly Slower
Wastes Space	Saves Space
Gender, PIN	Name, Address

Example
PIN Code
PIN CHAR(6)
Student Name
Name VARCHAR(100)

---

3. Date and Time Data Types
These store dates and time values.
DATE
Stores only the date.
Format
YYYY-MM-DD
Example
DOB DATE
Value
2005-06-15
Uses

- Date of Birth
- Joining Date

TIME
Stores only time.
Format
HH:MM:SS
Example

09:30:00
Uses

- Lecture Time
- Office Timing

DATETIME
Stores both date and time.
Example
2026-07-05 10:30:00
Uses

- Online Order Time
- Appointment Schedule

TIMESTAMP
Stores date and time.
Automatically updates when a record changes.
Example
CreatedAt TIMESTAMP
Uses

- Login Time
- Last Updated Record
- Audit Logs

YEAR
Stores only the year.
Example
2026
Uses

- Admission Year
- Graduation Year

Difference Between DATE and DATETIME

DATE	DATETIME
Stores only date	Stores date and time
Less storage	More storage
Birthday	Transaction Time

---

4. Boolean and Binary Data Types

BOOLEAN
Stores logical values.
Possible values

TRUE
FALSE
Internally stored as
1
0
Example
IsActive BOOLEAN

BIT
Stores binary values.
Example
Permission BIT(1)
Used for

- Access Rights
- Flags
- Switches

BLOB
BLOB stands for **Binary Large Object**.
Stores
Images
Videos
Audio
PDF Files
Documents
Example
Photo BLOB

Comparison of Major Data Types

Category	Example Data Types	Example Use
Numeric	INT, FLOAT, DECIMAL	Age, Salary
Character	CHAR, VARCHAR, TEXT	Name, Address
Date & Time	DATE, TIME, DATETIME	DOB, Login Time
Boolean	BOOLEAN, BIT	Active Status
Binary	BLOB	Images, Documents

Real-World Student Database Example

```
CREATE TABLE Student
(
StudentID INT,
StudentName VARCHAR(100),
Gender CHAR(1),
```

DOB DATE,
Email VARCHAR(100),
Fees DECIMAL(8,2),
IsActive BOOLEAN
);

Explanation

- StudentID → Whole Number
- StudentName → Variable Length Text
- Gender → Single Character
- DOB → Date
- Email → Text
- Fees → Decimal Value
- IsActive → TRUE/FALSE

Best Practices for Choosing Data Types

- Use the smallest suitable data type.
- Use VARCHAR for names and addresses.
- Use CHAR for fixed-length values like gender or PIN.
- Use DECIMAL for money.
- Use FLOAT only for approximate values.
- Store dates using DATE or DATETIME, not as text.
- Avoid using TEXT for short strings.

Common Mistakes

❌ Using FLOAT for salary.

✓ Use DECIMAL.

❌ Using TEXT for names.

✓ Use VARCHAR(100).

❌ Storing dates as strings.

"15/07/2026"

✓ Use

DATE

❌ Using BIGINT for age.

✓ Use INT or SMALLINT.

Key Points to Remember

- A **data type** defines the type of value a column can store.
- Choosing the correct data type improves performance, accuracy, and storage efficiency.

- **Numeric types** store numbers (INT, DECIMAL, FLOAT).
- **Character types** store text (CHAR, VARCHAR, TEXT).
- **Date and Time types** store dates and timestamps (DATE, TIME, DATETIME, TIMESTAMP).
- **Boolean and Binary types** store logical values and binary data (BOOLEAN, BIT, BLOB).
- Use **DECIMAL** for financial values, **VARCHAR** for variable-length text, and **CHAR** for fixed-length values.

Introduction to SQL Operator

In SQL, simply retrieving all records from a table is often not enough. We usually need to perform calculations, compare values, combine conditions, or search for specific patterns. SQL provides **operators** for these tasks.

For example, if we want to display students who scored more than 75 marks, SQL uses the **greater than (>) operator**. If we want to calculate a student's total marks, SQL uses the **addition (+) operator**.

Therefore, SQL operators are an essential part of writing powerful and meaningful SQL queries.

What is an SQL Operator?

Definition

An SQL operator is a symbol or keyword used to perform operations on one or more values in a SQL statement.

Operators help us:

- Perform mathematical calculations.
- Compare values.
- Combine multiple conditions.
- Search for patterns.
- Filter records.

Why Do We Need SQL Operators?

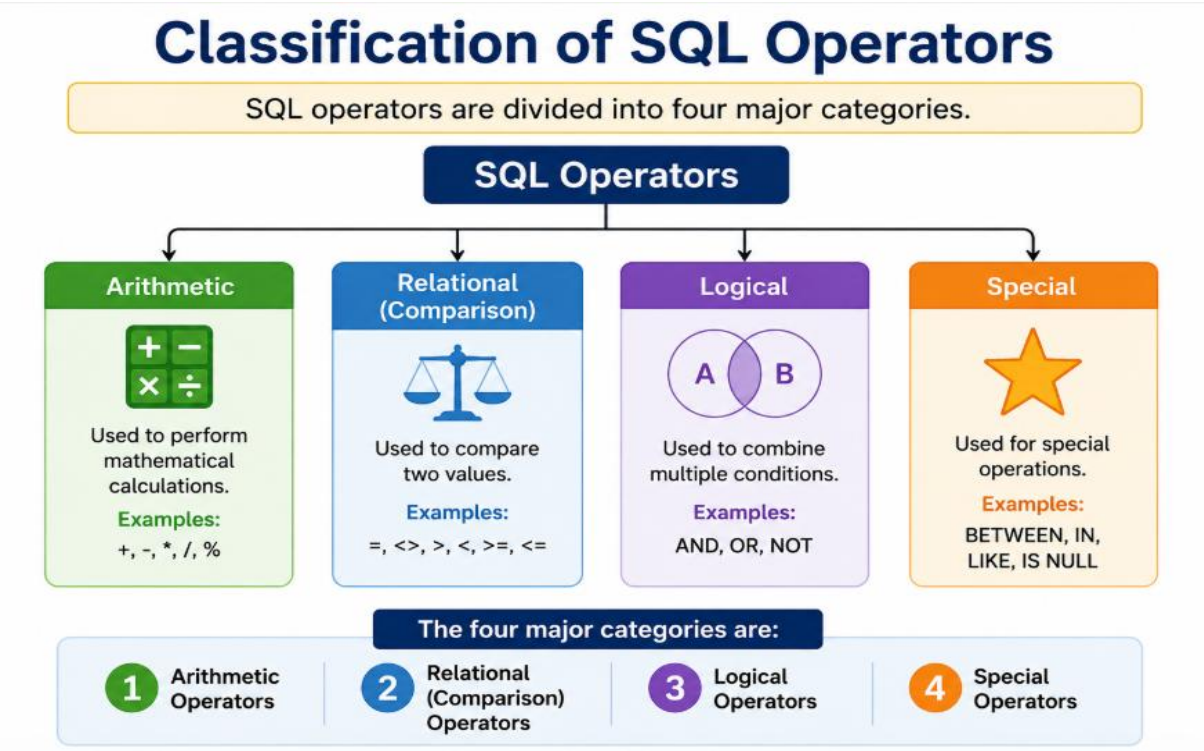
Imagine a college database containing thousands of student records.

The principal asks:

- Show students scoring more than 80 marks.
- Show students from the BCA department.
- Show students whose names start with "R".

These conditions are not possible without SQL operators.

Classification of SQL Operators



---

1. Arithmetic Operators

Arithmetic operators perform mathematical calculations.

They are mainly used with numeric data.

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus (Remainder)

Addition (+)

Used to add two numbers.

Example

SELECT 20 + 10;

Output

30

Real-Life Example

Calculate bonus marks.

SELECT Marks + 5

FROM Student;

If Rahul scored 80,

Output = 85

Subtraction (-)
Used to subtract one value from another.
Example
SELECT 90 - 10;
Output
80
Example
SELECT Salary - Tax
FROM Employee;

Multiplication (*)
Used to multiply numbers.
Example
SELECT Price * Quantity
FROM Product;
Suppose,
Price = ₹200
Quantity = 5
Output
₹1000

Division (/)
Used to divide values.
Example
SELECT 100/5;
Output
20

Modulus (%)
Returns the remainder after division.
Example
SELECT 15 % 4;
Output
3
Uses

- Checking even and odd numbers.
- Cyclic calculations.

Real-Life Example of Arithmetic Operators
Suppose an employee earns ₹50,000.
Annual Salary
SELECT Salary*12
FROM Employee;

Output
₹6,00,000

---

2. Relational (Comparison) Operators

Relational operators compare two values.
The result is always:

- TRUE
- FALSE

These operators are mainly used in the **WHERE** clause.

Types of Relational Operators

Operator	Meaning
=	Equal To
>	Greater Than
<	Less Than
>=	Greater Than or Equal To
<=	Less Than or Equal To
<> or !=	Not Equal To

Equal To (=)

Example
SELECT *
FROM Student
WHERE Marks=80;
Output
Displays students scoring exactly 80 marks.

Greater Than (>)

Example
SELECT *
FROM Student
WHERE Marks>75;
Output
Students scoring above 75.

Less Than (<)

Example
SELECT *
FROM Student
WHERE Age<20;
Displays students younger than 20.

Greater Than or Equal To (>=)

Example

```
SELECT *
FROM Student
WHERE Marks>=40;
```

Displays all passing students.

Less Than or Equal To (<=)

Example

```
SELECT *
FROM Employee
WHERE Salary<=30000;
```

Not Equal To (<>)

Example

```
SELECT *
FROM Student
WHERE Department<>'BCA';
```

Displays students from departments other than BCA.

Real-Life Example

Teacher wants students who passed.

Passing Marks = 40

Query

```
SELECT *
FROM Student
WHERE Marks>=40;
```

---

3. Logical Operators

Logical operators combine two or more conditions.

They are useful when multiple conditions must be checked together.

Types

Operator	Meaning
AND	Both conditions must be true
OR	At least one condition must be true
NOT	Reverses the condition

AND Operator

Both conditions must be TRUE.

Example

```
SELECT *
FROM Student
WHERE Marks>75
AND Department='BCA';
```

Output

Students who

✓ scored above 75

AND

✓ belong to BCA.

OR Operator

At least one condition must be TRUE.

Example

```
SELECT *
FROM Student
WHERE City='Jaipur'
OR City='Indore';
```

Output

Students from Jaipur or Indore.

NOT Operator

Reverses a condition.

Example

```
SELECT *
FROM Student
WHERE NOT City='Delhi';
```

Output

Students who are **not** from Delhi.

Difference Between AND and OR

AND	**OR**
Both conditions true	One condition true
Returns fewer records	Returns more records

Example

```
WHERE Age>18
AND City='Jaipur'
```

Only students satisfying both conditions.

---

4. Special Operators

Special operators perform advanced filtering.

IN Operator

Checks multiple values.
Instead of
WHERE City='Jaipur'
OR City='Delhi'
OR City='Indore'
Use
WHERE City IN
('Jaipur','Delhi','Indore');
Much shorter and easier.

BETWEEN Operator
Checks a range.
Example
SELECT *
FROM Student
WHERE Marks BETWEEN 60 AND 80;
Output
Students scoring between 60 and 80.

LIKE Operator
Searches text using patterns.
Wildcards

Symbol	Meaning
%	Any number of characters
_	Single character

Example 1
Names starting with A
SELECT *
FROM Student
WHERE Name LIKE 'A%';
Example
Amit
Anjali
Akash

Example 2
Names ending with a
SELECT *
FROM Student
WHERE Name LIKE '%a';

Example 3
Names containing "ra"

```
SELECT *
FROM Student
WHERE Name LIKE '%ra%';
```

IS NULL

Finds empty values.

Example

```
SELECT *
FROM Student
WHERE Email IS NULL;
```

Shows students whose email has not been entered.

EXISTS

Checks whether a subquery returns records.

Example

```
SELECT *
FROM Student
WHERE EXISTS
(
SELECT *
FROM Result
WHERE Student.RollNo=Result.RollNo
);
```

If matching records exist,
Output = TRUE.

Comparison of Special Operators

Operator	Purpose
IN	Match multiple values
BETWEEN	Check a range
LIKE	Pattern matching
IS NULL	Check empty values
EXISTS	Check subquery result

Arithmetic vs Relational Operators

Arithmetic	Relational
Performs calculations	Compares values
Returns numeric result	Returns TRUE/FALSE

Arithmetic	Relational
+, -, *, /	>, <, =

Relational vs Logical Operators

Relational	Logical
Compare values	Combine conditions
>, <, =	AND, OR, NOT

Operator Precedence

When multiple operators are used in one query, SQL follows a priority order.

Highest Priority

1. Parentheses ( )
2. Arithmetic Operators
3. Relational Operators
4. NOT
5. AND
6. OR

Example

```
SELECT *
FROM Student
WHERE Marks>60
AND Department='BCA'
OR City='Jaipur';
```

SQL evaluates

1. Marks > 60
2. Department = 'BCA'
3. AND
4. OR

To avoid confusion, use parentheses.

```
SELECT *
FROM Student
WHERE
(Marks>60
AND Department='BCA')
OR City='Jaipur';
```

Real-Life Examples

Student Database

```
SELECT *
FROM Student
WHERE Marks>75;
```

High-scoring students.

Employee Database
SELECT *
FROM Employee
WHERE Salary BETWEEN
30000 AND 50000;
Employees earning between ₹30,000 and ₹50,000.

Library Database
SELECT *
FROM Books
WHERE Author
LIKE 'R%';
Books written by authors whose names start with **R**.

Common Mistakes
❌ Using = with NULL
WHERE Email=NULL
✓ Correct
WHERE Email IS NULL;

❌ Wrong range
BETWEEN 80 AND 60
✓ Correct
BETWEEN 60 AND 80

❌ LIKE without wildcards
LIKE 'Rahul'
Behaves like =.
Use
LIKE 'Rah%'

❌ Mixing AND and OR without parentheses.
Always use parentheses for complex conditions.

Key Points to Remember

- SQL operators perform calculations, comparisons, and logical operations.
- **Arithmetic operators** (+, -, *, /, %) perform mathematical calculations.
- **Relational operators** (=, >, <, >=, <=, <>) compare values and return TRUE or FALSE.
- **Logical operators** (AND, OR, NOT) combine or modify conditions.

- **Special operators** (IN, BETWEEN, LIKE, IS NULL, EXISTS) simplify advanced filtering.
- Use parentheses to control the order of evaluation in complex conditions.
- IS NULL should always be used to check for missing values.

Expression Introduction

In SQL, we often need to perform calculations, compare values, combine conditions, or manipulate text while retrieving data. To achieve this, SQL uses **Expressions**.

An SQL expression combines **columns, values, operators, and functions** to produce a single result.

For example, if we want to calculate the annual salary of an employee from the monthly salary, we can write:

```
SELECT Salary * 12 AS Annual_Salary
FROM Employee;
```

Here, **Salary * 12** is an SQL expression.

Thus, expressions make SQL queries more powerful and flexible.

What is an SQL Expression?

Definition

An SQL Expression is a combination of one or more values, columns, operators, and functions that SQL evaluates to produce a single value.

Expressions are used in:

- SELECT clause
- WHERE clause
- HAVING clause
- ORDER BY clause
- UPDATE statement

Why Do We Need SQL Expressions?

Suppose a company stores only the monthly salary of employees.

Instead of storing the annual salary separately, SQL can calculate it whenever required.

Example

```
SELECT Salary*12
FROM Employee;
```

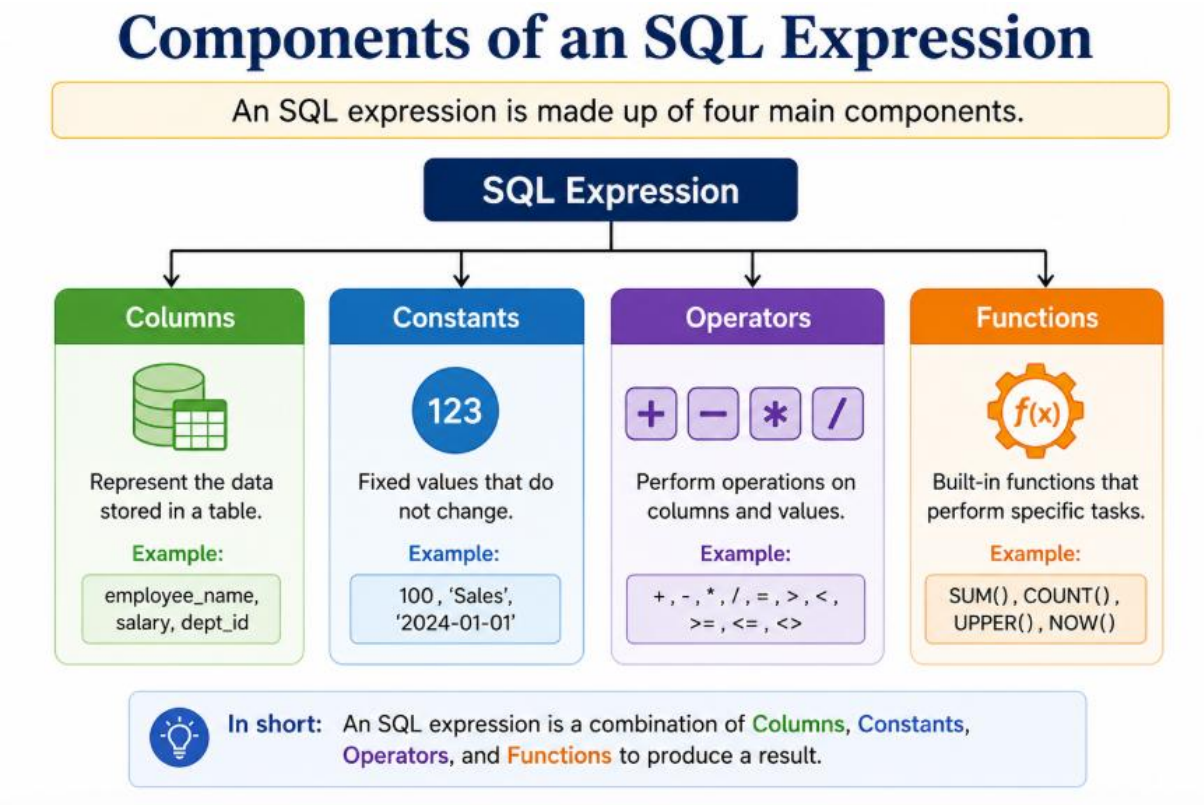
Similarly,

A teacher wants to add 5 bonus marks.

```
SELECT Marks+5
FROM Student;
```

SQL Expressions perform these calculations automatically.

Components of an SQL Expression



---

1. Columns

Columns are fields of a table.

Example

Salary

Marks

Age

Name

Example Query

SELECT Salary

FROM Employee;

Here,

Salary is a column.

---

2. Constants (Literal Values)

Constants are fixed values.

Examples

100

'Rahul'

3.14

TRUE

Example
SELECT Salary+1000
FROM Employee;
Here,
1000 is a constant.

---

3. Operators
Operators perform operations.
Examples
Arithmetic
+
-
*
/
%
Comparison
=
>
<
Logical
AND

OR

NOT

---

4. Functions
Functions perform predefined operations.
Examples
UPPER()

LOWER()

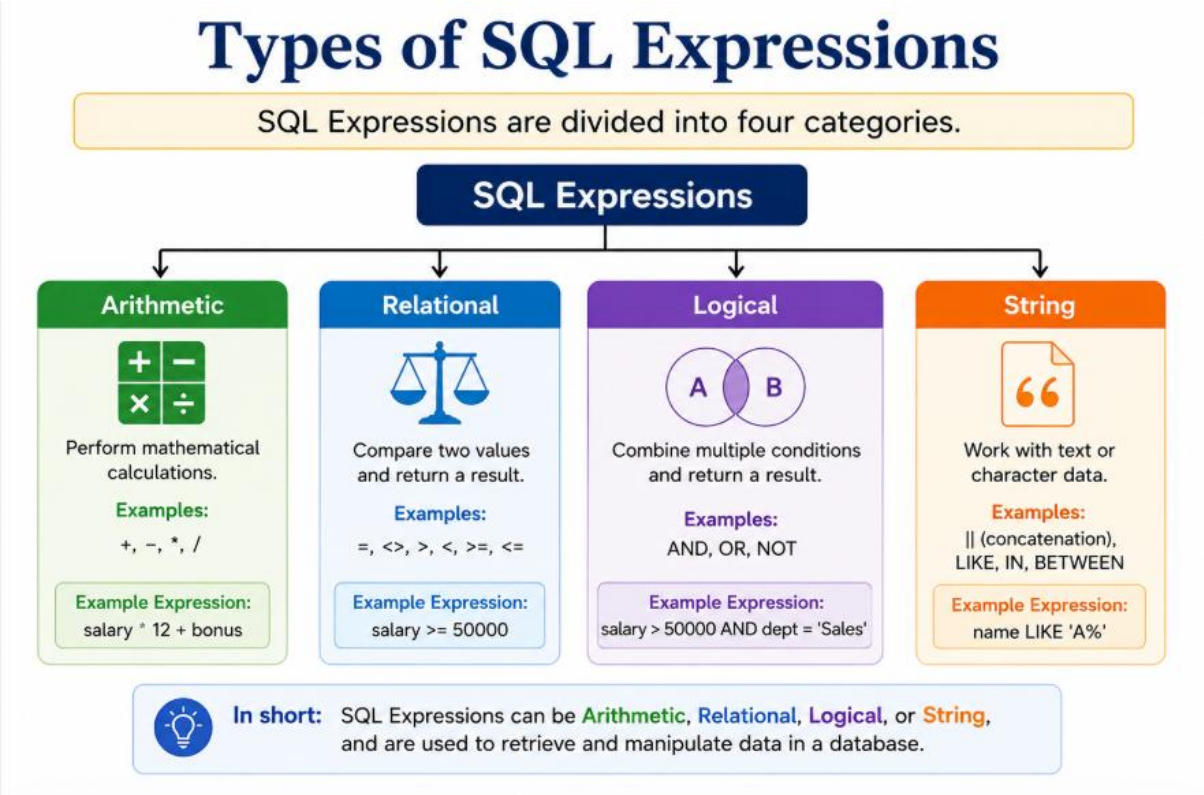
ROUND()

NOW()

LENGTH()

COUNT()
Example
SELECT UPPER(Name)
FROM Student;

Types of SQL Expressions



---

1. Arithmetic Expressions

Arithmetic expressions perform mathematical calculations.

Operators used

+

-

*

/

%

Example 1

Calculate Annual Salary

SELECT Salary*12 AS AnnualSalary

FROM Employee;

Monthly Salary

₹30,000

Annual Salary

₹3,60,000

Example 2

Discount Price

SELECT Price-(Price*0.10)

FROM Product;
Price
₹500
Discount
10%
Result
₹450

Example 3
Modulus
SELECT 17%5;
Output
2

---

2. Relational Expressions
Relational expressions compare two values.
Output
TRUE
FALSE

Example
SELECT *
FROM Student
WHERE Marks>=40;
Students scoring 40 or more marks are selected.

Another Example
SELECT *
FROM Employee
WHERE Salary<25000;
Only employees earning less than ₹25,000 are displayed.

---

3. Logical Expressions
Logical expressions combine multiple conditions.
Operators

- AND
- OR
- NOT

Example
SELECT *
FROM Student
WHERE Marks>60

AND Department='BCA';
Both conditions must be TRUE.

Example
SELECT *
FROM Employee
WHERE City='Jaipur'
OR City='Indore';
Either condition may be TRUE.

Example
SELECT *
FROM Student
WHERE NOT City='Delhi';
Displays students who are not from Delhi.

---

4. String Expressions
String expressions manipulate text.
Common Functions

- CONCAT()
- UPPER()
- LOWER()
- LENGTH()
- LIKE

CONCAT()
Joins two strings.
Example
SELECT CONCAT(FirstName,' ',LastName)
AS FullName
FROM Student;
Output
Rahul Sharma

UPPER()
Converts text into uppercase.
Example
SELECT UPPER(Name)
FROM Student;
Rahul
↓
RAHUL

LOWER()
Converts text into lowercase.
Example
SELECT LOWER(Name)
FROM Student;
RAHUL
↓
rahul

LENGTH()
Finds the number of characters.
Example
SELECT LENGTH(Name)
FROM Student;
Name
Rahul
Output
5

SQL Operator Precedence
When multiple operators are used in one expression, SQL follows a predefined order.
This is called **Operator Precedence**.

Order of Precedence
Highest Priority

1. Parentheses ( )
2. Multiplication (*)
3. Division (/)
4. Modulus (%)
5. Addition (+)
6. Subtraction (-)
7. Comparison Operators
8. NOT
9. AND
10. OR

Lowest Priority

Example Without Parentheses
SELECT 10+5*2;
SQL first calculates
5 × 2 = 10
Then
10 + 10 = 20

Output
20

Example With Parentheses
SELECT (10+5)*2;
SQL first calculates
10+5=15
Then
15×2=30
Output
30

Why Parentheses are Important
Consider
SELECT *
FROM Student
WHERE Marks>60
AND Department='BCA'
OR City='Jaipur';
This query may return unexpected results.
Correct Query
SELECT *
FROM Student
WHERE
(Marks>60
AND Department='BCA')
OR City='Jaipur';
Parentheses remove confusion.

Expression Evaluation
Example
Salary+500*2>10000
AND Department='IT'
SQL evaluates

1. Multiplication

500×2
↓

2. Addition

Salary+1000
↓

3. Comparison

10000
↓

4. Logical AND

Final Result

Real-World Examples

Student Database

SELECT Name,
Marks+5 AS BonusMarks
FROM Student;
Adds 5 grace marks.

Employee Database

SELECT Name,
Salary*12 AS AnnualSalary
FROM Employee;
Calculates annual salary.

Banking System

SELECT AccountNo,
Balance*0.05
AS Interest
FROM Account;
Calculates 5% interest.

Shopping Website

SELECT ProductName,
Price-(Price*0.20)
AS DiscountPrice
FROM Product;
Calculates 20% discount.

Common Mistakes

1. Comparing NULL with =

Wrong
WHERE Salary=NULL;
Correct
WHERE Salary IS NULL;

---

2. Mixing Data Types

Wrong
Salary+Name
Correct
Use compatible data types only.

---

3. Forgetting Parentheses

Wrong

```
Marks>60
OR Age>18
AND City='Jaipur'
```

Correct

```
(Marks>60
OR Age>18)
AND City='Jaipur'
```

---

4. Forgetting Operators

Wrong

```
Marks 50
```

Correct

```
Marks>50
```

Best Practices

- Use parentheses in complex expressions.
- Use meaningful aliases using **AS**.
- Keep expressions simple.
- Test expressions before using them in large queries.
- Avoid mixing incompatible data types.
- Write readable SQL statements with proper indentation.

Difference Between Operator and Expression

Operator	**Expression**
A symbol used to perform an operation.	A complete combination of values, operators, and functions.
Example: +	Example: Salary + Bonus
Performs one operation.	Produces a final result.

Key Points to Remember

- An SQL expression combines **columns, constants, operators, and functions** to produce a single result.
- Expressions can be **Arithmetic, Relational, Logical, or String**.
- Arithmetic expressions perform calculations, while relational expressions compare values.
- Logical expressions combine conditions using **AND**, **OR**, and **NOT**.
- String expressions manipulate text using functions like **CONCAT()**, **UPPER()**, **LOWER()**, and **LENGTH()**.
- SQL follows a fixed order of evaluation called **operator precedence**.
- Parentheses should be used to make expressions clear and avoid unexpected results.

- Expressions are commonly used in **SELECT**, **WHERE**, **HAVING**, and **ORDER BY** clauses.

Introduction of SQL* Plus:

After learning SQL, SQL commands, data types, operators, and expressions, the next step is to understand **SQL*Plus**.

Many beginners think **SQL** and **SQL*Plus** are the same, but they are different.

- **SQL** is a language used to communicate with a database.
- **SQL*Plus** is a software tool provided by Oracle that allows users to write, execute, and manage SQL and PL/SQL commands.

Think of SQL as the **language**, while SQL*Plus is the **environment (tool)** used to execute that language.

What is SQL*Plus?

Definition

SQL*Plus is a command-line interface (CLI) tool provided by Oracle that allows users to execute SQL and PL/SQL statements and interact with an Oracle Database.

It acts as a bridge between the user and the Oracle Database.

Why Do We Need SQL*Plus?

Suppose you write the following SQL statement:

SELECT * FROM Student;

This statement cannot execute by itself.

It needs a software environment that:

- Accepts the SQL command.
- Sends it to the Oracle Database.
- Receives the result.
- Displays the output.

SQL*Plus performs all these tasks.

Working of SQL*Plus

Components of SQL*Plus

SQL*Plus consists of the following components:

1. User

The person who writes SQL or PL/SQL commands.

Example:

SELECT * FROM Student;

---

2. SQL*Plus Interface

Accepts commands from the user.

It also provides:

- Command prompt
- Report formatting
- Script execution
- Error messages

---

3. Oracle Database

The actual database where data is stored.

Oracle processes the SQL commands received from SQL*Plus.

Features of SQL*Plus

SQL*Plus provides many useful features.

1. Executes SQL Statements

Example

```
SELECT * FROM Student;
```

Displays all student records.

---

2. Executes PL/SQL Programs

Example

```
BEGIN
DBMS_OUTPUT.PUT_LINE('Welcome');
END;
/
```

Runs PL/SQL blocks.

---

3. Executes SQL Script Files

Suppose we have

student.sql

SQL*Plus can execute the file.

```
START student.sql;
```

or

```
@student.sql
```

---

4. Saves SQL Commands

Example

```
SAVE student.sql
```

Stores commands in a file.

---

5. Edits SQL Statements

Example

```
EDIT
```

Opens the current SQL statement in an editor.

---

6. Formats Query Results

SQL*Plus allows users to change:

- Column width
- Heading
- Page size
- Line size

Example

```
COLUMN Name FORMAT A20
```

---

7. Generates Reports

Example

Employee Salary Report
Student Result Report
Attendance Report

---

8. Displays Error Messages
Example
SELECT FORM Student;
Output
ORA-00923:
FROM keyword not found where expected
The error helps users identify mistakes.

SQL*Plus Environment
A typical SQL*Plus session looks like this:
SQL>
This prompt indicates that SQL*Plus is ready to accept SQL commands.
Example
SQL> SELECT * FROM Student;

Connecting to Oracle Database
To use SQL*Plus, users must first connect to an Oracle database.
Syntax
sqlplus username/password
Example
sqlplus system/admin123
After successful login,
SQL>
appears.

Disconnecting from Oracle Database
Command
EXIT;
or
QUIT;
Terminates the SQL*Plus session.

Common SQL*Plus Commands
CONNECT
Connects to the database.
CONNECT system/admin123;

DISCONNECT
Disconnects the current session.

DISCONNECT;

EXIT
Leaves SQL*Plus.
EXIT;

HELP
Displays help information.
HELP;

DESCRIBE (DESC)
Displays table structure.
Example
DESC Student;
Output
Name Type

ROLLNO NUMBER

NAME VARCHAR2

MARKS NUMBER

LIST
Displays the current SQL command stored in memory.
LIST;

CLEAR SCREEN
Clears the screen.
CLEAR SCREEN;

SAVE
Saves SQL commands into a file.
SAVE student.sql;

START
Runs a saved SQL script.
START student.sql;

@ Command
Also executes SQL files.
@student.sql

SQL*Plus Commands Summary

Command	Purpose
CONNECT	Connect to database
DISCONNECT	Disconnect database
EXIT	Exit SQL*Plus
HELP	Display help
DESC	Display table structure
LIST	Show current command
SAVE	Save SQL file
START	Run SQL script
@	Execute SQL file

SQL vs SQL*Plus

SQL	SQL*Plus
Query Language	Oracle Tool
ANSI Standard	Oracle Product
Used with all RDBMS	Used with Oracle
Performs database operations	Executes SQL and PL/SQL
Cannot execute scripts	Executes SQL script files
No formatting features	Supports report formatting

Advantages of SQL*Plus

- Easy to use.
- Executes SQL and PL/SQL.
- Supports scripting.
- Generates formatted reports.
- Displays detailed error messages.
- Useful for database administration.
- Supports batch processing.
- Lightweight command-line tool.

Limitations of SQL*Plus

- Works mainly with Oracle Database.
- Command-line interface is less user-friendly than graphical tools.
- Limited visualization capabilities.
- Not suitable for advanced database design.
- Requires knowledge of Oracle commands.

SQL*Plus vs MySQL Workbench

SQL*Plus	MySQL Workbench
Oracle Tool	MySQL Tool
Command Line	Graphical Interface (GUI)
Oracle Database	MySQL Database
Text-Based	Visual Environment
Better for scripting	Better for beginners

Real-Life Example

Suppose a university wants to display all student records.

The user writes

SELECT * FROM Student;

The process is

User

Writes SQL Query

SQL*Plus

Sends Query

Oracle Database

Processes Data

SQL*Plus

Displays Student Records

Common Mistakes

1. Wrong Username or Password

ORA-01017

Invalid username/password

Always verify login credentials.

---

2. Forgetting Semicolon

Wrong

SELECT * FROM Student

Correct

SELECT * FROM Student;

---

3. Running Script Without Path

Wrong

@student.sql

If the file is not in the current directory.

Correct

@C:\SQL\student.sql

---

4. Incorrect Table Name

Wrong

DESC Students;

If the table name is actually Student.

Always verify object names before execution.

Best Practices

- Connect to the correct database before running commands.
- End SQL statements with a semicolon (;).
- Save important SQL scripts for future use.
- Use DESC to understand table structures before querying.
- Read and understand error messages instead of ignoring them.
- Keep SQL scripts organized in folders.
- Log out properly using EXIT or QUIT.

Case Study 1: College Student Management System

Scenario:

ABC College maintains a database to store student information such as Roll Number, Name, Department, Semester, Marks, Email, and Date of Birth. The administrator wants to create the database structure before entering student records.

Questions

1. Which category of SQL commands should be used to create the Student table?
2. Suggest appropriate data types for Roll Number, Name, Marks, Email, and Date of Birth.
3. Which SQL command would you use to add a new column named **Mobile Number**?
4. Explain why VARCHAR is more suitable than CHAR for storing student names.

Case Study 2: Employee Payroll System

Scenario:

XYZ Company stores employee details including Employee ID, Name, Department, Monthly Salary, and Bonus. The HR department wants to calculate the annual salary of each employee.

Questions

1. Which SQL operator is required to calculate annual salary?
2. Write an SQL expression to calculate annual salary.

3. Which data type is most suitable for storing salary? Justify your answer.
4. Why is DECIMAL preferred over FLOAT for salary?

Case Study 3: Hospital Management System

Scenario:

A hospital database stores patient records such as Patient ID, Name, Blood Group, Date of Admission, and Status (Admitted/Discharged).

Questions

1. Which data type should be used for the Date of Admission?
2. Which data type is suitable for storing patient status (TRUE/FALSE)?
3. Write an SQL query to display patients admitted after **1 January 2026**.
4. Which SQL command category is used to retrieve patient records?

Case Study 4: Library Management System

Scenario:

A library stores Book ID, Book Name, Author, Price, and Quantity. The librarian wants to find books written by authors whose names start with the letter **R**.

Questions

1. Which SQL operator should be used to search author names?
2. Write the SQL query for this requirement.
3. Which SQL command retrieves all books costing more than ₹500?
4. Explain the difference between the LIKE operator and the = operator.

Case Study 5: Online Shopping Website

Scenario:

An online shopping website stores Product ID, Product Name, Price, Discount Percentage, and Stock Quantity.

Questions

1. Write an SQL expression to calculate the discounted price.
2. Which arithmetic operators are used in this calculation?
3. Which data type is appropriate for Product Price?
4. Explain why arithmetic expressions are useful in SQL.

Case Study 6: Banking System

Scenario:

A bank stores Account Number, Customer Name, Balance, Account Type, and Last Transaction Date. The manager wants to display customers whose balance is between ₹10,000 and ₹50,000.

Questions

1. Which SQL operator is most suitable for this requirement?
2. Write the SQL query.
3. Which SQL command category is used to retrieve customer information?
4. Which data type should be used for the balance amount?

Case Study 7: University Examination System

Scenario:

The university stores marks of students. The examination department wants to display students who scored more than 75 marks and belong to the BCA department.

Questions

1. Which logical operator should be used?
2. Write the SQL query.
3. What will happen if the OR operator is used instead of AND?
4. Explain the difference between relational operators and logical operators.

Case Study 8: School Admission Portal

Scenario:

The admission office stores student details. Some students have not yet provided their email addresses.

Questions

1. Which SQL operator is used to find records with missing email addresses?
2. Write the SQL query.
3. Why is IS NULL used instead of = NULL?
4. Which SQL command category is used to retrieve these records?

Case Study 9: Railway Reservation System

Scenario:

The railway reservation system allows passengers to book tickets. After booking, payment is completed successfully. If the payment fails, all booking changes should be cancelled.

Questions

1. Which SQL command permanently saves the booking?
2. Which SQL command cancels all changes if payment fails?
3. What is the purpose of SAVEPOINT?
4. Which category of SQL commands handles these operations?

Case Study 10: Oracle Database Administration

Scenario:

A database administrator is using SQL*Plus to manage an Oracle database. The administrator needs to connect to the database, check the structure of the Student table, execute a saved SQL script, and then exit the application.

Questions

1. Which SQL*Plus command is used to connect to the database?
2. Which command displays the structure of the Student table?
3. Which command executes a saved SQL script?
4. Which command is used to exit SQL*Plus?
5. Explain the difference between **SQL** and **SQL*Plus** in this scenario.